



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

ARM PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A IoT & Edge

TOTAL: 10 QUESTIONS

Q1 What is ARM and what problem in IoT & Edge does it solve? **Model**

Q6 How does ARM handle reliability and high availability? **Observability**

Q2 What are the core components or concepts in ARM? **Architecture**

Q7 What observability does ARM provide out of the box? **Lifecycle**

Q3 How does ARM compare to its main alternatives? **Configuration**

Q8 What are common performance bottlenecks in ARM? **Compliance**

Q4 How do you get started with ARM for a new project? **Hardening**

Q9 What security best practices apply when running ARM? **Testing**

Q5 What configuration options most affect ARM's behavior in production? **IO / IdP**

Q10 What mistakes do teams commonly make when adopting ARM? **Tuning**



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 ARM is a tool or platform that

Mitigation

ARM is a tool or platform that automates or simplifies a specific concern in the IoT & Edge domain, reducing manual effort and operational complexity for engineering teams.

A6 ARM provides HA through leader election, replication, and observability

ARM provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 ARM has key building blocks that work

Primitives

ARM has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 ARM exposes metrics (Prometheus endpoint or built-in dashboard) and automation

ARM exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 ARM trades off simplicity against feature richness

Hardening

ARM trades off simplicity against feature richness differently than its alternatives. Choose ARM when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install ARM via its package manager or

Gotchas

Install ARM via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run ARM with the principle of least

Assessment

Run ARM with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

Concurrency

A10 Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.